8gent Code

Never hit usage caps again™

> Terminal-first agentic coding with 40%+ token savings through AST-first exploration.

License MIT

Built with Bun

Token Savings 40%+

The Problem

AI coding tools are hitting usage limits because they read entire files when they only need specific functions. This wastes tokens and money.

```
❌ Traditional: search → read entire file → guess → patch
✅ 8gent:       plan → retrieve symbol → compose → verify
```

The Solution

8gent extracts symbols from your codebase using AST parsing, so the AI only sees what it needs.

```
# Instead of reading a 2,119 token file...
8gent outline src/parser.ts

# ...get just the 61 tokens you need:
8gent symbol src/parser.ts::buildSymbolId
```

Result: 97% less tokens, 97% less cost.

Benchmarks

Real results from 8gent's own codebase:

Metric	Traditional	AST-First	Savings
Average file read	1,027 tokens	546 tokens	**46.8%**
Symbol retrieval	2,119 tokens	61 tokens	**97.1%**
10K operations (Claude Opus)	$3,084	$1,640	**$1,444 saved**

Run `8gent benchmark` to see results on your codebase.

Quick Start

```
# Install
bun add -g 8gent-code

# Or clone and run locally
git clone https://github.com/8gent/8gent-code.git
cd 8gent-code
bun install

# Get file outline (symbols, functions, classes)
bun run bin/8gent.ts outline src/index.ts

# Get specific symbol source
bun run bin/8gent.ts symbol src/utils.ts::parseDate

# Search across codebase
bun run bin/8gent.ts search "handleError" --kinds=function
```

The Name

8gent combines two ideas:

- **8** → infinity (∞ rotated)
- **gent** → gentleman / agent

An **infinite agent**: a disciplined system that grows without increasing prompt size.

Philosophy

Most coding agents navigate codebases like this:

```
search → read → guess → patch
```

8gent works differently:

```
plan → retrieve → compose → verify
```

Structure replaces brute-force exploration.

Architecture

```
User Intent
    ↓
8gent CLI / TUI
    ↓
Planner (task decomposition)
    ↓
Toolshed (capability discovery)
    ↓
AST Index (symbol extraction)
    ↓
Codebase
```

Core Packages

Package	Purpose
`ast-index`	TypeScript Compiler API for native TS/JS parsing
`planner`	Task planning with dependency tracking
`toolshed`	Dynamic tool registration and discovery
`types`	Shared TypeScript definitions

Toolshed

The Toolshed is the capability layer. Tools register themselves and are discovered dynamically.

```
registerTool({
  name: "get_outline",
  description: "Get all symbols in a file without loading full content",
  capabilities: ["code", "code.symbol"],
  permissions: ["read:code"],
}, async (input, context) => {
  // AST-first implementation
});
```

Available tools:

- `get_outline` - File symbol extraction
- `get_symbol` - Single symbol retrieval
- `search_symbols` - Cross-file symbol search

Project Structure

```

```

```
📂 8gent-code
├── bin/
│   └── 8gent.ts           # CLI entry point
├── apps/
│   └── tui/               # Terminal UI (Ink/React)
├── packages/
│   ├── ast-index/         # AST parsing (TS Compiler API)
│   ├── planner/           # Task planning engine
│   ├── toolshed/          # Tool registry and discovery
│   ├── types/             # Shared TypeScript types
│   └── workflow/          # Workflow execution
└── scripts/
    ├── benchmark.ts       # Full benchmark suite
    └── demo-savings.ts    # Token savings demo
```

Commands

```
8gent init              # Initialize in current directory
8gent outline <file>    # Get symbol outline
8gent symbol <id>       # Get symbol source code
8gent search <query>    # Search for symbols
8gent benchmark         # Run efficiency benchmarks
8gent demo              # Show token savings demo
```

Acknowledgments

The AST-first approach was influenced by jcodemunch, which demonstrated the power of symbol-level code retrieval for reducing token consumption. 8gent builds on this foundation with a complete toolshed architecture.

Contributing

1. Fork the repo
2. Create your feature branch (`git checkout -b feature/amazing`)
3. Run benchmarks to verify savings (`bun run benchmark`)
4. Commit your changes
5. Push to the branch
6. Open a Pull Request

License

MIT © James Spalding

Stop paying for tokens you don't need.

⭐ Star us on GitHub